

Documentacao do Projeto - LocaJogos Univap

Tema e objetivo

O tema escolhido foi um Sistema Web de Locadora de Jogos, atendendo video games e board games. O objetivo e permitir o cadastro de usuarios, categorias e jogos, alem do controle de emprestimos, devolucoes, estoque, relatorios, importacao/exportacao de dados e logs.

Regras de negocio principais

- Um usuario pode possuir varios emprestimos.
- Uma categoria pode classificar varios jogos.
- Um emprestimo pode possuir varios jogos, e um jogo pode aparecer em varios emprestimos.
- A relacao N:N entre emprestimos e jogos e resolvida pela tabela `items\_emprestimo`.
- Ao criar um emprestimo, o estoque dos jogos e reduzido automaticamente.
- Ao devolver um emprestimo, o estoque e restaurado automaticamente.
- Se a devolucao ocorrer apos a data prevista, a multa e calculada em R\$ 2,00 por dia e por item.
- Nao e permitido emprestar jogo sem estoque suficiente.
- As rotas privadas exigem token JWT.

Arquitetura MVC + Service Layer

O projeto foi separado nas camadas solicitadas pelo enunciado:

- `routes`: mapeia URLs e verbos HTTP.
- `middlewares`: autentica, valida, registra logs e trata erros.
- `controllers`: recebem a requisicao e retornam JSON padronizado.
- `services`: concentram regras de negocio.
- `daos`: executam consultas SQL preparadas e acesso ao MongoDB.
- `models`: representam entidades do dominio.
- `config`: centraliza configuracoes de MySQL, MongoDB e ambiente.

Fluxo:

Request -> Router -> Middleware -> Controller -> Service -> DAO -> Banco -> Response

Interfaces

Como JavaScript nao possui interfaces nativas iguais a Java/C#, foram criadas classes didaticas que funcionam como contratos:

- `IDAO`
- `IService`
- `IController`

As classes concretas estendem esses contratos e implementam os metodos esperados, como `listar`, `buscarPorId`, `criar`, `atualizar` e `remover`.

Services

Principais services:

- `AuthService`: login, logout, bcrypt e JWT.
- `UsuarioService`: CRUD de usuarios e hash de senha.
- `CategoriaService`: CRUD de categorias.
- `JogoService`: CRUD de jogos e upload de imagem.
- `EmprestimoService`: criacao, devolucao, multa e regras de estoque.
- `DashboardService`: dados agregados para graficos.
- `ExportacaoService`: exportacao/importacao JSON.
- `LogService`: logs no MongoDB e exportacao XML.
- `RelatorioService`: PDFs com PDFKit.

Banco MySQL

Tabelas:

- `usuarios`
- `categorias`
- `jogos`
- `emprestimos`
- `itens\_emprestimo`

Relacionamentos:

- `usuarios` 1:N `emprestimos`
- `categorias` 1:N `jogos`
- `emprestimos` N:N `jogos` por `itens\_emprestimo`

O script `database/banco.sql` cria o banco, tabelas, chaves primarias, chaves estrangeiras, indices e inserts de teste.

MongoDB

Os logs ficam na colecao `logs`, com campos:

- `usuario`
- `endpoint`
- `metodo`
- `timestamp`
- `status\_code`
- `acao`
- `ip`
- `detalhes`

Eventos registrados:

- login/logout
- acesso a rotas
- inclusao de registros
- alteracao de dados
- exclusao de dados
- erros e excecoes

Exportacao XML

A rota `GET /api/logs/exportar/xml` consulta os logs no MongoDB e usa `xml2js` para gerar XML com estrutura organizada. Existem filtros opcionais por usuario e data.

Relatorios PDF

O projeto usa PDFKit para gerar:

- Relatorio de jogos
- Relatorio de emprestimos

Os PDFs possuem titulo, data/hora, usuario que gerou, tabela e totais.

Grafico

O dashboard usa Chart.js. Os dados vem do MySQL pela rota `GET /api/dashboard/resumo`, mostrando jogos mais alugados e metricas administrativas.

Importacao e exportacao JSON

Rotas:

- `GET /api/exportacao/:entidade`
- `POST /api/importacao/:entidade`

As entidades permitidas sao:

- usuarios
- categorias
- jogos
- emprestimos

A importacao recebe arquivo pelo frontend usando `input type="file"`.

Como executar

1. Rodar `npm install`.
2. Copiar `.env.example` para `.env`.
3. Executar `database/banco.sql` no MySQL.
4. Iniciar o MongoDB.
5. Rodar `npm start`.
6. Acessar `http://localhost:3000`.

Usuario de teste:

admin@locajogos.com
123456

Endpoints

Principais rotas:

- `POST /api/auth/login`
- `POST /api/auth/logout`
- `GET /api/usuarios`
- `POST /api/usuarios`
- `PUT /api/usuarios/:id`
- `DELETE /api/usuarios/:id`
- `GET /api/categorias`
- `POST /api/categorias`
- `PUT /api/categorias/:id`
- `DELETE /api/categorias/:id`
- `GET /api/jogos`
- `POST /api/jogos`
- `PUT /api/jogos/:id`
- `DELETE /api/jogos/:id`
- `GET /api/emprestimos`
- `POST /api/emprestimos`
- `PUT /api/emprestimos/:id`
- `POST /api/emprestimos/:id/devolver`
- `DELETE /api/emprestimos/:id`
- `GET /api/dashboard/resumo`
- `GET /api/exportacao/:entidade`
- `POST /api/importacao/:entidade`
- `GET /api/logs`
- `GET /api/logs/exportar/xml`
- `GET /api/relatorios/jogos/pdf`
- `GET /api/relatorios/emprestimos/pdf`